Overview

When writing and analyzing quantum programs, we need to understand how different parts of the program affect each other — these relationships are called **dependencies**. In classical programming, you're probably already familiar with the idea that one line of code can affect another (e.g., you have to set a variable before you use it). Quantum programs have all of those same dependencies, *plus* several new ones that arise from the unique rules of quantum mechanics.

This page defines the four quantum-specific dependency types that are captured in our **Quantum Program Dependence Graph (QPDG)**, and explains how the QPDG is used to enable **quantum program slicing** — a technique for isolating only the parts of a program relevant to a particular computation.

> **Source:** https://ieeexplore.ieee.org/document/11029533

Why Do Quantum Programs Need New Dependency Types?

In classical programs, two primary dependency types are sufficient:

- **Control dependencies** — whether a statement executes depends on the outcome of a condition (e.g., an `if` statement).
- **Data dependencies** — a statement reads a value that was written by a previous statement.

Quantum programs introduce physical constraints that these two categories cannot capture. Phenomena like entanglement, qubit decoherence, and probabilistic measurement each create relationships between statements that have no classical equivalent. Ignoring these would make any analysis — such as slicing or debugging — semantically incorrect.

The Four Quantum Dependency Types

1. Entanglement Dependence

What is entanglement? Entanglement is a quantum mechanical phenomenon where two or more qubits become linked such that the state of one qubit cannot be described independently of the other(s). Once entangled, measuring one qubit *instantly* determines what the measurement of the other qubit will be — even if the qubits are physically separated. This is one of the most distinctive and counterintuitive aspects of quantum mechanics.

What is entanglement dependence? Entanglement dependence captures the relationship between program statements that operate on entangled qubits. If statement `u` entangles two qubits, then statement `v` that later measures or operates on either of those qubits is *entanglement-dependent* on `u`. The outcome at `v` is directly shaped by what happened at `u`.

Example: A common two-qubit entangled state is the Bell state, created by:

1. Applying a **Hadamard gate** (`H`) to qubit 0, placing it in superposition (equal chance of measuring 0 or 1).
2. Applying a **CNOT gate** between qubit 0 (control) and qubit 1 (target), which entangles them.

```
qc.h(0)       # Puts qubit 0 in superposition
qc.cx(0, 1)   # Entangles qubit 0 and qubit 1
qc.measure_all()
```

After this, measuring the qubits will always yield either `00` or `11` — never `01` or `10`. The `measure_all()` statement is entanglement-dependent on both `qc.h(0)` and `qc.cx(0,1)` because those two statements jointly created the shared state being measured.

Why it matters for analysis: Any slicing or analysis tool must track entanglement across statements. If you remove a statement that created an entangled state, the behavior of downstream measurements changes entirely — even though there may be no classical data flow connecting them.

---

2. Measurement Dependence

What is quantum measurement? Measuring a qubit is not a passive read operation like in classical computing. It is a physical interaction that **collapses** the qubit from a superposition of states into a single definite classical value (0 or 1). This collapse is irreversible and probabilistic — the same qubit in the same state can yield different results on different runs.

What is measurement dependence? Measurement dependence captures the relationship between a measurement statement and any subsequent operations whose execution is *conditioned on the measurement result*. Because measurement outcomes are probabilistic, the control flow of a quantum program can branch in ways determined at runtime — a property with no direct classical analogue.

Example:

```
# qreg represents the quantum register
# creg represents the classical register

qc.h(qreg[0])                          # Put qubit into superposition
qc.measure(qreg[0], creg[0])           # Measure it; result stored in classical register
qc.x(qreg[0]).c_if(creg, 1)           # Apply X gate only if measurement result was 1
```

Here, whether the X gate executes at all depends on the outcome of the measurement. The X gate statement is measurement-dependent on the `measure` statement. If you sliced out the measurement, the conditional operation would have nothing to branch on.

Why it matters for analysis: Any analysis tool must propagate the effects of measurement results through the program's control flow. Slicing criteria involving post-measurement operations must include the measurement itself — otherwise the slice is incomplete and the control flow is broken.

---

3. Phase Dependence

#TODO

---

4. Qubit Data Dependence

#TODO

---

5. Classical Dependence (Data and control (should really be its own page))

#TODO

Quantum Program Dependence Graph (QPDG)

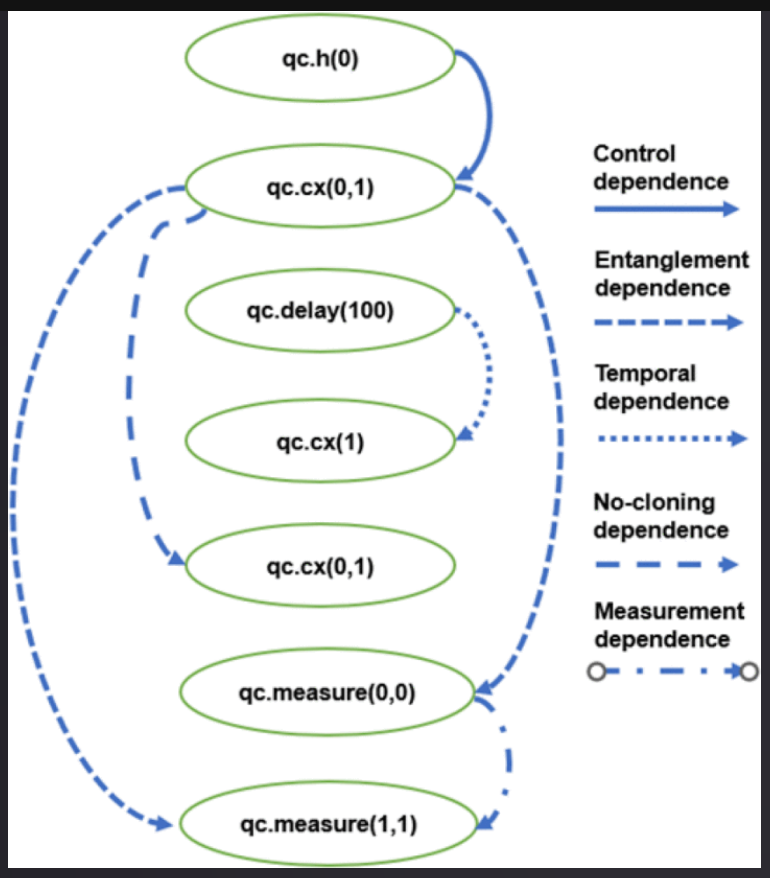
The **Quantum Program Dependence Graph (QPDG)** is a formal representation of all dependencies — both classical and quantum — within a quantum program. It is an **arc-classified directed graph** where:

- **Nodes** represent individual program statements.
- **Edges** represent dependencies between statements, labelled by dependency type (control, data, entanglement, or measurement).

The QPDG extends the classical Program Dependence Graph (PDG) by adding the four quantum-specific dependency types described above. This makes it possible to reason about quantum programs in a rigorous and complete way, capturing relationships that a classical PDG would miss entirely.

The QPDG is foundational to several software engineering tasks for quantum programs, including:

- **Program slicing** — isolating parts of a program relevant to a specific computation (see below).
- **Debugging** — tracing unexpected results back to their source.
- **Testing** — verifying that operations produce intended quantum states.
- **Optimization** — identifying operations that can be safely reordered or removed.



Application: Quantum Program Slicing

What is quantum program slicing?

Quantum program slicing is a technique for extracting a *subset* of a quantum program — called a **slice** — that contains only the statements relevant to a particular variable or computation at a particular point in the program. This is useful when you want to focus your analysis or debugging effort on a specific part of the program's behavior without wading through unrelated code.

Slicing is defined in terms of a **slicing criterion**: a pair `(s, V)`, where `s` is a statement in the program and `V` is a set of variables of interest at that statement.

The two types of static quantum slices

Slice Type	Criterion	Includes
Static backward slice `SS(s, V)`	`(s, V)` where `V` are variables *used* at `s`	All statements that could affect the values of `V` at `s`
Static forward slice `SFS(s, V)`	`(s, V)` where `V` are variables *defined* at `s`	All statements that could be affected by the values of `V` at `s`

How does the QPDG enable slicing?

Using the QPDG, both backward and forward slicing reduce to **graph reachability problems**:

- A **backward slice** is computed by finding all nodes that can *reach* `s` by following dependency edges.
- A **forward slice** is computed by finding all nodes *reachable from* `s` via dependency edges.

This makes slicing efficient and well-defined, because the QPDG explicitly encodes all the dependency relationships — including the quantum-specific ones — that determine what's relevant to any given computation.

Why do quantum-specific dependencies matter for slicing?

A slice that only respects classical control and data dependencies might look correct but produce wrong results when run on a quantum computer. For example:

- Removing a statement that created an entangled state (entanglement dependence) would change measurement outcomes in ways not captured by data flow alone.
- Omitting a measurement (measurement dependence) could sever a conditional branch that downstream operations rely on.

By building slices from the QPDG, we ensure that the resulting slice is not just *syntactically smaller*, but **semantically equivalent** to the original program with respect to the variables of interest.

Potential dependencies currently out of scope

1. No-Cloning Dependence

What is the no-cloning theorem? The no-cloning theorem is a fundamental result in quantum mechanics stating that it is *impossible* to create an exact, independent copy of an arbitrary unknown quantum state. This is in stark contrast to classical computing, where copying a bit's value is trivial. In quantum computing, any attempt to "copy" a qubit's state instead *entangles* the two qubits — it does not produce an independent duplicate.

What is no-cloning dependence? No-cloning dependence arises when a quantum program attempts to reuse or share a qubit's state across multiple statements. This dependency type tracks such attempts to ensure the program doesn't inadvertently assume that true copying has occurred, which would violate quantum mechanics and lead to incorrect logic.

Example:

```
qc.x(0)       # Initialize state of qubit 0
qc.cx(0, 1)   # Attempt to "copy" qubit 0's state into qubit 1
qc.measure_all()
```

This looks like a copy operation to a classical programmer, but it actually creates an entanglement between qubit 0 and qubit 1. Qubit 1's state is now *conditional* on qubit 0 — not an independent copy of it. Any statement downstream that treats qubit 1 as a free, independent clone of qubit 0 is making a mistake.

Why it matters for analysis: Slicing and analysis tools must recognise that shared qubit usage across statements doesn't mean the states are duplicated — it means they're correlated. Failing to model this correctly can produce slices that appear valid but are semantically broken.

Why it falls outside our scope: Functionally, no-cloning dependence doesn't need to be destinct from entanglement dependence. No-cloning is simply a rule of quantum systems, not something unique that must be represented for slicing purposes.

---

2. Temporal Dependence

What is qubit coherence? Qubits are fragile. They can only maintain their quantum state (superposition, entanglement, etc.) for a limited amount of time before environmental interference causes **decoherence** — essentially, the quantum information is lost and the qubit behaves classically. This window of time is called the **coherence time**.

What is temporal dependence? Temporal dependence captures the constraint that certain quantum operations must be executed in a specific order *and within a specific timeframe* to produce correct results. Unlike classical programs where reordering independent statements is often safe, in quantum programs, delays or reordering can cause qubits to decohere before they are used, corrupting the result.

Example:

```
qc.h(0)           # Put qubit 0 in superposition
qc.delay(500, 0)  # Simulate a delay (qubit may decohere during this time)
qc.x(0)           # Apply X gate — but has the qubit already decohered?
qc.measure_all()
```

Whether the X gate has the intended effect depends entirely on whether the qubit survived the delay without decohering. Additionally, in a Bell state circuit, the gates `qc.h(0)` and `qc.cx(0,1)` *must* be applied before `qc.measure_all()` — measuring prematurely collapses the quantum state before entanglement is established, giving meaningless results.

Why it matters for analysis: Temporal dependence ensures that operations are not reordered or omitted by an analysis tool in ways that violate coherence constraints. A slice that removes timing-critical operations, or reorders them, can produce a syntactically valid but physically incorrect program.

Why it falls outside our scope: Decoherence time is inconsistent across quantum systems. As a result, there is no definitive way to determine whether or not a qubit will decohere at a certain point for any system.

Comments